

Node.js – Övningar Dag 2

Det går också bra att byta ut node till nodemon om man vill i alla övningar.

Övning 1

Då det blir bättre struktur kommer vi att använda en egen module att ha vår connection till MySQL i övningarna (som i 3-node-mysql-version2). Då är det bra att ha gjort något enkelt exempel med att göra en egen module först.

Titta på exemplet 1-node-mymodule.

1. Skapa en ny mapp: node-dag2-ovn1

2. Kör i terminalen i den nya mappen:

```
npm init
```

```
npm install express
```

3. Skapa en fil som heter app.js där du bygger upp kod för en grundläggande Express-applikation. Du kan titta i exemplet node-mymodule.

4. Du ska göra en egen module som t ex gör en uträkning eller skriver ut en text.

5. Nu ska du använda din module i app.js genom att skriva t ex

```
const addition = require('./mymodule');
```

dvs ./ (som innebär att filen som i exemplet ovan heter mymodule ligger i samma mapp som app.js).

Därefter skriver du ut innehållet från modulen genom t ex:

```
app.get('/', (req, res) => {
```

```
  res.send("Summan är " + addition)
```

```
});
```

Övning 2

Utgå från exemplet 3-node-mysql-version2

Det krävs att du har följt instruktioner.txt till exemplet och skapat en ny databas med tabeller för bok och kategori.

1. Skapa en ny mapp: node-dag2-ovn2

2. Kör i terminalen i den nya katalogen:

```
npm init
```

```
npm install express
```

```
npm install mysql
```

```
npm install cors
```

3. Kopiera över filerna `connectionMySQL.js` och `app.js` samt public-katalogen från exemplet `node-mysql-ver2`
4. Nu ska du göra endpoints med tillhörande Requests för kategori dvs `get`, `post`, `put`, `delete` så att vi har CRUD-funktionalitet även för kategori-tabellen.

Övning 3

Nu ska du bygga en egen applikation med en MySQL-databas utifrån exemplet `3-node-mysql-version2`.

Bestäm ett tema t ex film eller spel som har två tabeller kategori och huvudtabellen t ex film. Skapa en ny databas och skriv `CREATE TABLE`-satser och `INSERT INTO` för att fylla databasen med några rader.

Börja sedan att skriva själva applikationen som ska ha CRUD-funktionalitet för t ex film och kategori. Du kan därefter bygga ut med andra endpoints t ex visa någon form av statistik kopplat till databasens tema.

När du är klar med själva backend dvs `server.js` kan du arbeta med public-katalogen och göra html-sidor som visar t ex film och kategori och html-sidor med formulär för att lägga till kategori respektive t ex film.

1. Skapa en ny katalog `node-dag2-ovn3`

2. Kör i terminalen i den nya katalogen:

```
npm init
npm install express
npm install mysql
npm install cors
```

3. Kopiera över filen `app.js` och `connectionMySQL.js` och modifiera i koden alternativt titta på uppbyggnaden av `app.js` och skriv koden steg-för-steg själv.

Jag försöker bryta ner vad koden ska göra (utifrån exemplet):

- Skriv `require` för det som behövs till applikationen (`express`, `cors`, din egen `connection`).
- Skriv `app.use` för `cors` samt `middleware` för `public-mappen`.
- Skriv `app.listen` för att få igång en server.
- Skriv alla endpoints kopplat till respektive Request dvs `get`, `post`, `put`, `delete`

Tänk på när du arbetar med applikationen/övningen:

- Testa alltid SQL-koden direkt mot databasen så du vet att den inte innehåller några fel.
- Testa alltid dina endpoints med respektive Request i Insomnia eller Postman så att allting fungerar som tänkt.

- Blir det fel i applikationen t ex att det är något fel med SQL-koden och applikationen stängs ner. Klipp då in SQL-koden du får från felmeddelandet i terminalen och kör den direkt mot databasen i Datagrip och se till så att SQL-koden fungerar där först. Sedan ändrar du i koden i app.js och testar igen.
- Sätt dig in i koden hur upplägget fungerar med felhantering.
- Arbeta därefter med Frontend dvs i public-mappen.
- Sammanfattningsvis är det i början man behöver hitta ett arbetssätt där man får rutin på att testa och felsöka då vi är beroende av SQL-koden mot databasen och koden vi skriver i node.js och därefter html och javascript i public.

Större övning: Gör en egen Backend med Node.js, Express och MySQL:

Utgå ifrån databasen för övningen med skivor i MySQL-delen i kursen alternativt den databas du använde för Labb 1.

Begränsa till delar av databasen så att det inte blir för stort utan mer får insyn om vilka steg man bygger upp allting i och lite hur omfattande arbetet är. Du kan sedan bygga vidare.

Ifall tid finns gör Frontend till delar av din Backend.